

Machine Learning

Lecture 6:

Gradient boosting

Iurii Efimov



girafe
ai

Outline

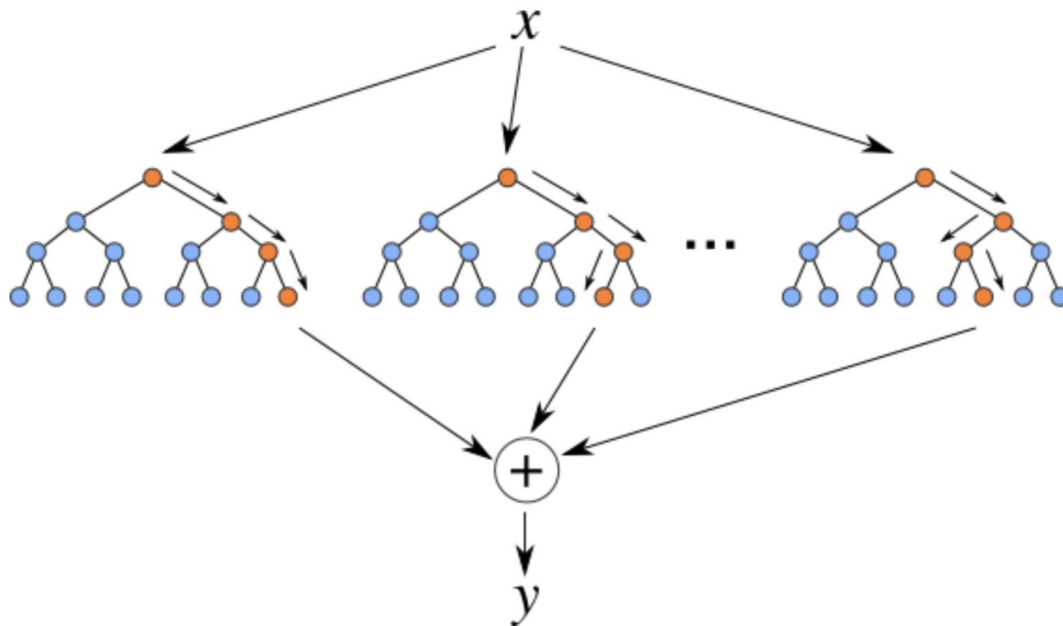
1. Intuitions
2. Gradient boosting theory
3. Examples
4. Libraries
5. Stacking & Blending

Based on: ml-mipt 2017 by Victor Kantor,
Evgeny Sokolov HSE 2018, mlcourse_open by ODS

Random Forest



Bagging + RSM = Random Forest



Random Forest

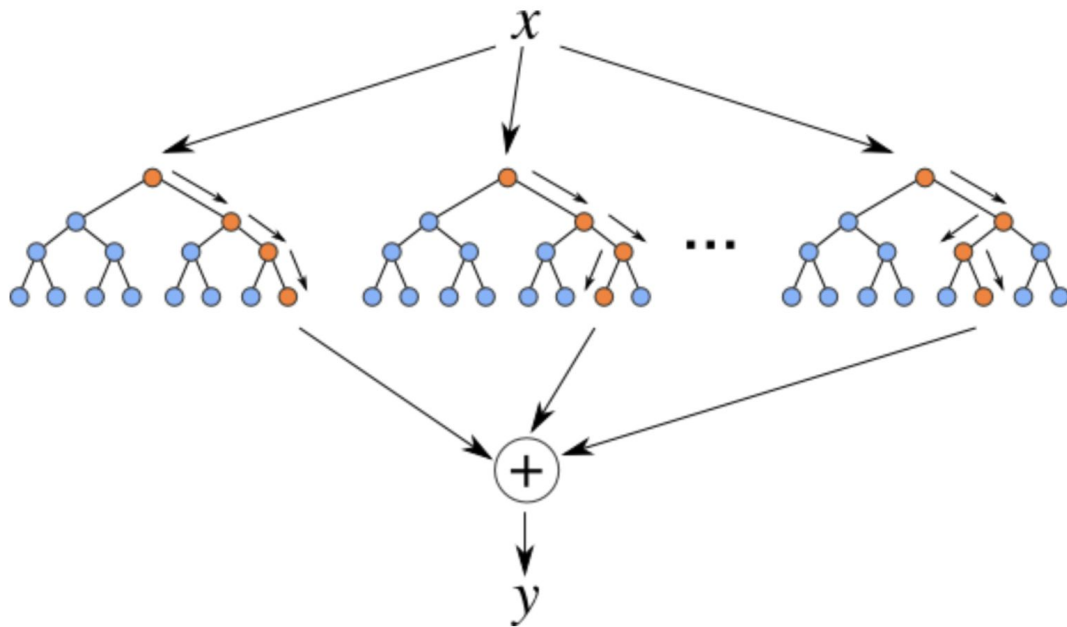


- One of the greatest “universal” models.
- There are some modifications: Extremely Randomized Trees, Isolation Forest, etc.

Random Forest



Is Random Forest decreasing bias or variance by building the trees ensemble?



Boosting

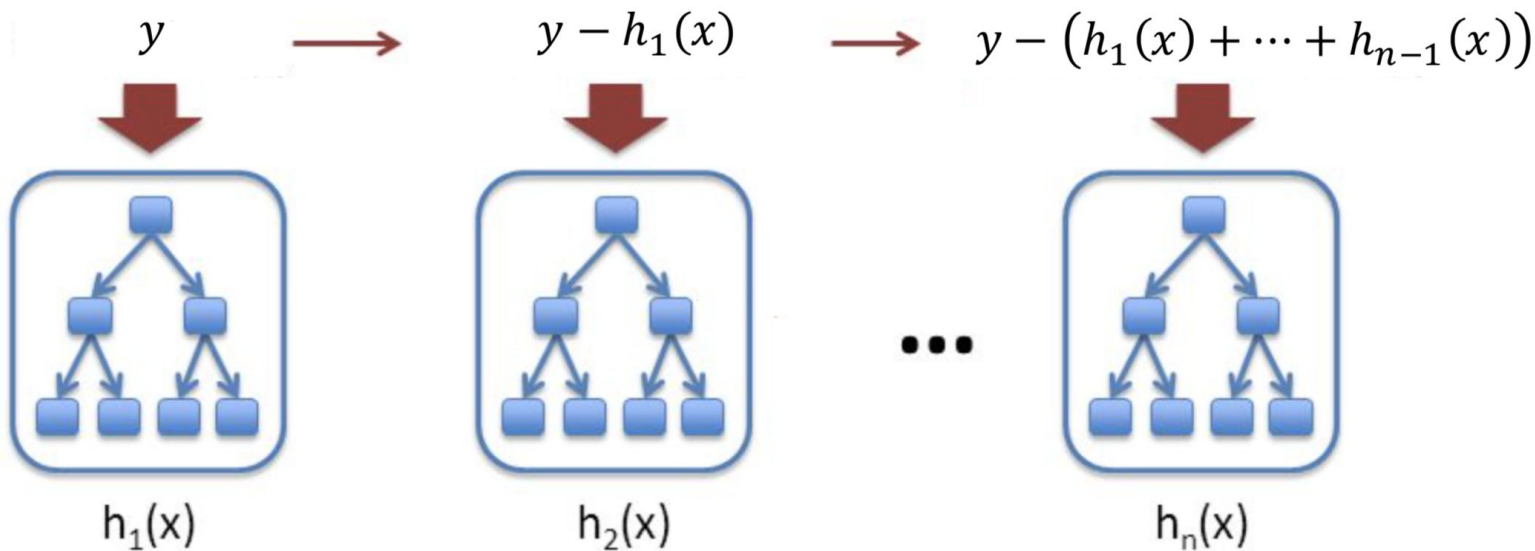
girafe
ai

01

Boosting for _MSE_



$$a_n(x) = h_1(x) + \dots + h_n(x)$$

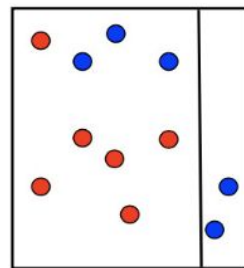
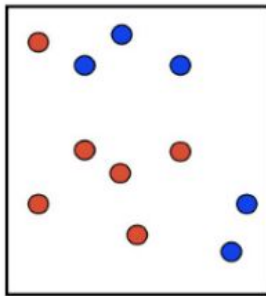


* only in case of MSE loss!!!

Boosting: intuition

Binary classification

Use decision stumps.



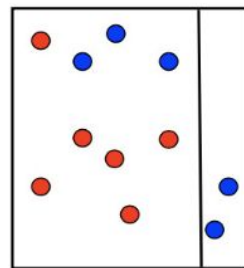
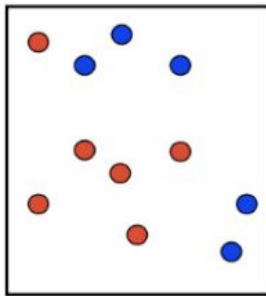
$t = 1$



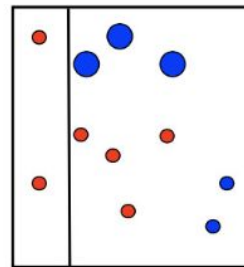
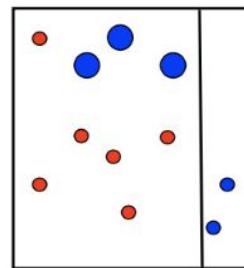
Boosting: intuition

Binary classification

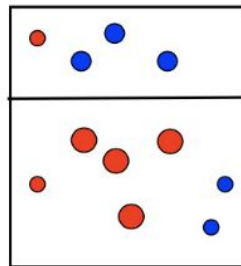
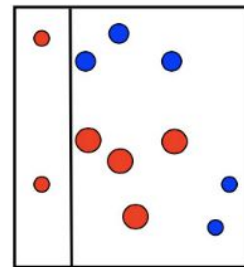
Use decision stumps.



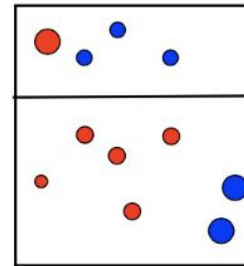
$t = 1$



$t = 2$



$t = 3$

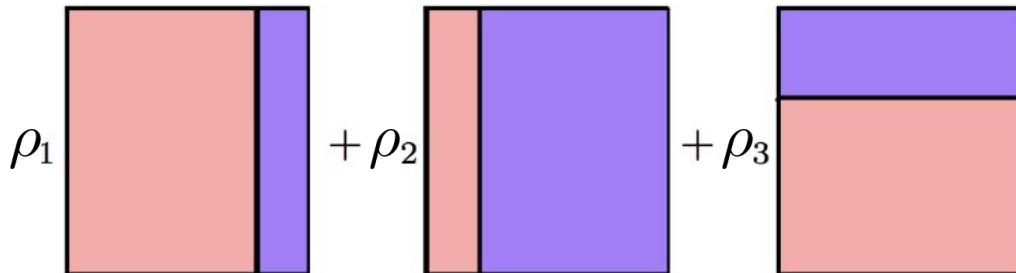
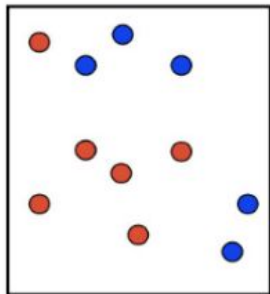


Boosting: intuition

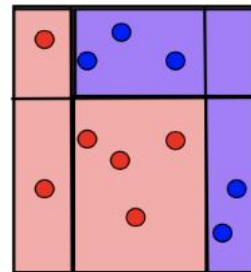


Binary classification

Use decision stumps.



$$\hat{f}_T(x) = \sum_{t=1}^T \rho_t h_t(x) =$$



Gradient boosting theory

girafe
ai

02

Model as vector of predictions



Let's assume we trained first model to predict original target, then we only care about this model's predictions on training dataset.

$$X = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} \in R^{n \times p} \quad Y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} = Y^* \in R^n \quad f_1 = \text{fit}(X, Y)$$

$$\bar{f}_1 = f_1(X) = \begin{pmatrix} f_1(x_1) \\ f_1(x_2) \\ \vdots \\ f_1(x_n) \end{pmatrix} \quad \text{How good is this model?} \quad L(y^*, \bar{f}_1) = L_{f_1}$$

Ensemble as vector of predictions



The same notation can be applied to an ensemble.

$$g_k(X) = \sum_{i=1}^k f_i(X)$$

$$g_k(X) = \begin{pmatrix} g_k(x_1) \\ g_k(x_2) \\ \vdots \\ g_k(x_n) \end{pmatrix}$$

How to extend ensemble?

$$\min_{\bar{g}_k} L(Y^*, \bar{g}_k)$$

Ideally: $Y^* = \bar{g}_k$

Ensemble extension method



We want to extend ensemble additively, so let's put that idea to formulas:

$$L(Y^*, \bar{g}_k) = L(Y^*, g_{k-1}^- + \bar{f}_k) \rightarrow \min_{\bar{f}_k}$$

Existing ensemble is fixed and we want to vary only the last term.

Usually L and y^* are complex, so we can't directly find such $\bar{f}_k$ that will minimize loss globally. But we can find $\bar{f}_k$ which will make situation a bit better than it was.

A small step towards y^* and minimal L value.

Which can be found by linearization of the expression with respect to $\bar{f}_k$.

Optimal local extension



Local linearization leads to taking first derivative:

$$\begin{aligned} \frac{\partial L(Y^*, \bar{g}_{k-1} + \bar{f}_k)}{\partial \bar{f}_k} &= \frac{\partial L(Y^*, \bar{p})}{\partial \bar{p}} \cdot \frac{\partial(\bar{g}_{k-1} + \bar{f}_k)}{\partial \bar{f}_k} = \\ &= \frac{\partial L(Y^*, \bar{p})}{\partial \bar{p}} = \partial L(\bar{g}_{k-1}) \end{aligned}$$

$\bar{p}$ here means whole
ensemble prediction

∂L is a vector!



Optimal local extension

Local linearization leads to taking first derivative:

$$\begin{aligned}\frac{\partial L(Y^*, \bar{g}_{k-1} + \bar{f}_k)}{\partial \bar{f}_k} &= \frac{\partial L(Y^*, \bar{p})}{\partial \bar{p}} \cdot \frac{\partial(\bar{g}_{k-1} + \bar{f}_k)}{\partial \bar{f}_k} = \\ &= \frac{\partial L(Y^*, \bar{p})}{\partial \bar{p}} = \partial L(\bar{g}_{k-1})\end{aligned}$$

$$\bar{g}_k = \bar{g}_{k-1} - \bar{f}_k \cdot \partial L(\bar{g}_{k-1})$$



Optimal local extension

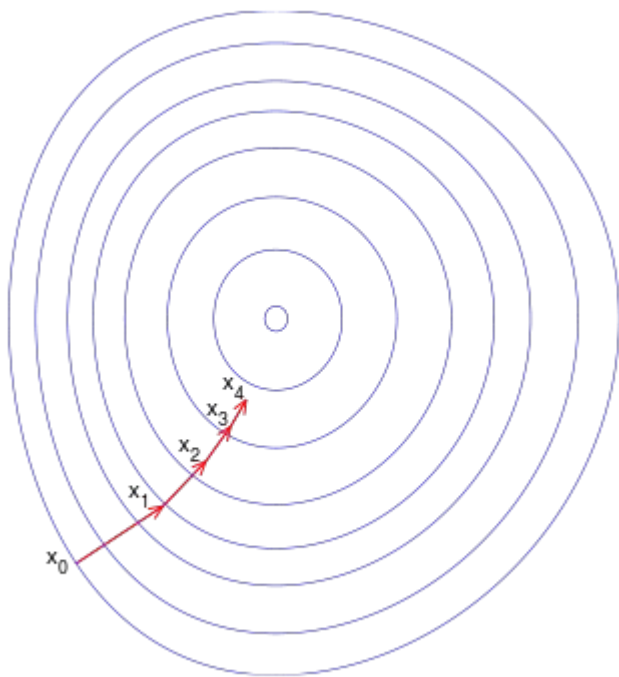
Local linearization leads to taking first derivative:

$$\begin{aligned}\frac{\partial L(Y^*, \bar{g}_{k-1} + \bar{f}_k)}{\partial \bar{f}_k} &= \frac{\partial L(Y^*, \bar{p})}{\partial \bar{p}} \cdot \frac{\partial(\bar{g}_{k-1} + \bar{f}_k)}{\partial \bar{f}_k} = \\ &= \frac{\partial L(Y^*, \bar{p})}{\partial \bar{p}} = \partial L(\bar{g}_{k-1})\end{aligned}$$

Gradient descent formula

$$\bar{g}_k = \bar{g}_{k-1} - lr \cdot \partial L(\bar{g}_{k-1})$$

Geometric interpretation



Vector space for all $\mathbf{f}, \mathbf{g}$ and $\mathbf{y}^*$ is a space where each coordinate i corresponds to an answer on the object x_i in training dataset.

And we are doing regular Gradient Descent in that space.

Next model predicts the best step towards right answer.

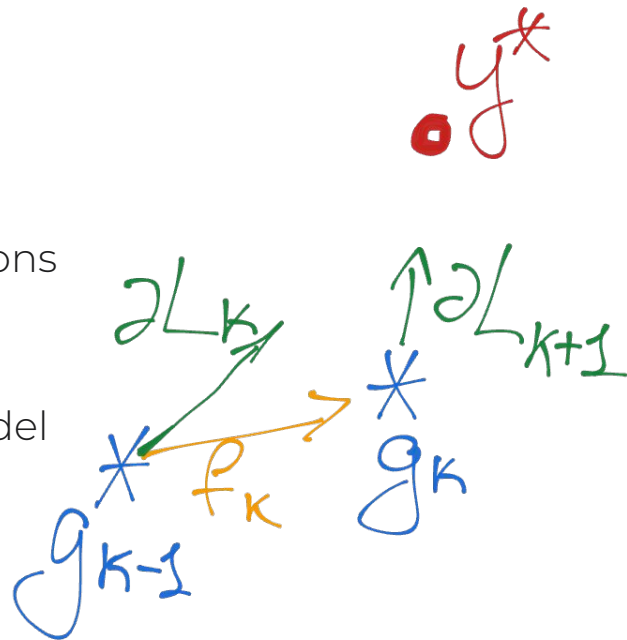
Basic model choice



Why not linear? Why not kNN? or others?

Why trees as basic algorithms?

1. GB doesn't need differentiable base model
 - a. target is given independently between iterations
2. Trees have high variance
 - a. gradient varies a lot between iterations
 - b. meaning high variance in target for basic model
3. Error on any step doesn't ruin whole algorithm
 - a. we recompute true gradients on each step
 - b. meta-algorithm (GB) fixes such errors!!!



So for now trees are the only option for GB



Important loss functions

Two most frequently used losses are MSE and Crossentropy

$$L_{mse}(Y, \bar{p}) = \frac{1}{n} \sum_{i=1}^n (y_i - \bar{p}(x_i))$$

$$\frac{\partial L_{mse}}{\partial \bar{p}_k} = \frac{1}{n} 2(\bar{p}_k - Y)$$



Important loss functions

Two most frequently used losses are MSE and Crossentropy

$$L_{ce}(Y^*, \bar{p}) = \frac{1}{n} \sum_{i=1}^n \sum_{j=1}^c [y_i = j] \log(p_k(x_i))$$

$$\frac{\partial L_{ce}}{\partial \bar{p}_k} = \frac{1}{n} \sum_{i=1}^n \sum_{j=1}^c [y_i = j] \frac{1}{\bar{p}_k(x_i)}$$



Other boostings

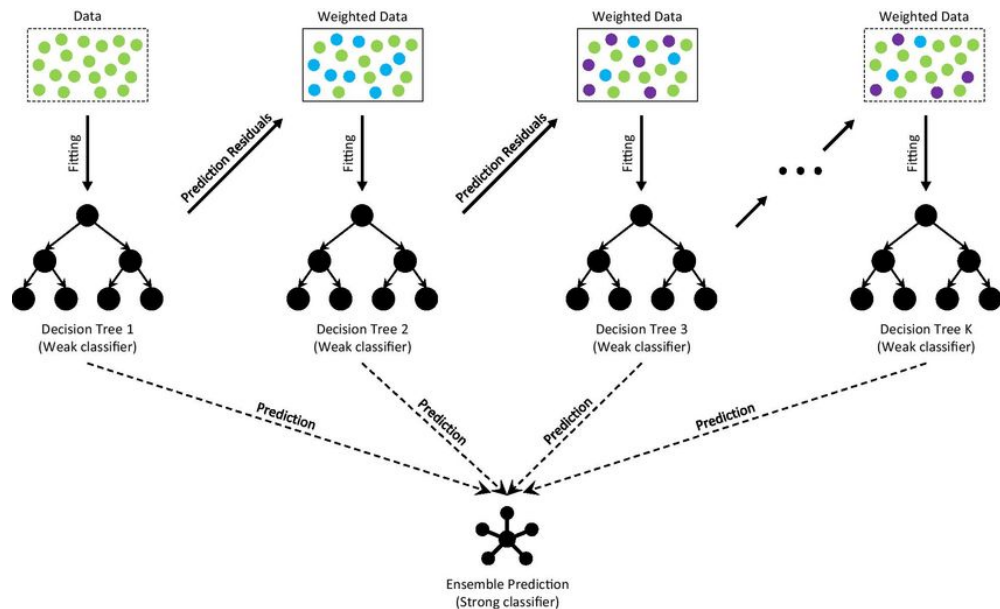
Historically there were a lot of boosting algorithms (where each next model fixes previous ensemble). Here is a list of selected algorithms:

- [AdaBoost](#) [1995]
- [LogitBoost](#)
- [BrownBoost](#)
- [CoBoosting](#)
- LPBoost
- TotalBoost

As of today Gradient Boosting overperformed them all.

So one more **Bitter lesson** for us

Parallelization



Unlike Bagging, training for GB is done by one tree and can not be parallelized.

But for inference you can run all GB trees in parallel and then sum up results.

	Training	Inference
Bagging	parallel	parallel
Boosting	sequential	parallel

Applicability



Now boostings are **used in production a lot**.

They shine bright on **tabular data** giving relatively light (in both training and inference) and accurate algorithm.

Neural networks barely can outperform GB in this case, and even in this case their costs become a problem (still people try to make breakthrough).

For so called unstructured data (images, texts, etc) it works not so well because of trees nature to treat each feature independently.

Summary



1. Gradient Boosting is a powerful meta-algorithm
2. Basic models are trained sequentially
3. Each basic model predicts gradient of loss function of previous ensemble
 - a. residuals of previous ensemble - only for MSE!
4. Basic model is always decision tree
5. Inference is done in parallel
6. GB is best for tabular data

Gradient boosting: beautiful demo



Great demo:

http://arogozhnikov.github.io/2016/06/24/gradient_boosting_explained.html

Gradient boosting examples

girafe
ai

03

Gradient boosting



What we need:

- Data
- Loss function and its gradient.
- Family of algorithms (with constraints if necessary).
- Number of iterations M .
- Initial value (GBM by Friedman): constant.



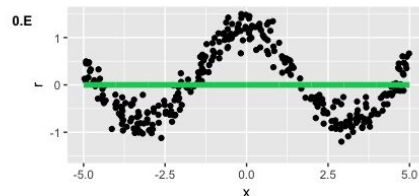
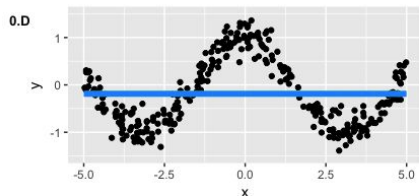
Gradient boosting: example

What we need:

- Data: toy dataset $y = \cos(x) + \epsilon, \epsilon \sim \mathcal{N}(0, \frac{1}{5}), x \in [-5, 5]$
- Loss function: MSE
- Family of algorithms: trees with depth 2
- Number of iterations $M = 3$
- Initial value: use mean value

Example by ODS; source: <https://habr.com/ru/company/ods/blog/327250/>

Gradient boosting: example

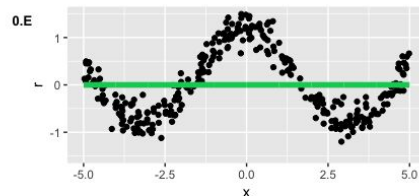
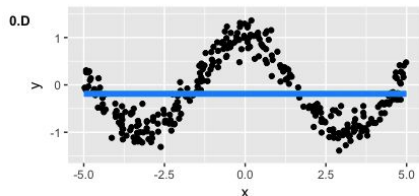


Left: full ensemble on each step.

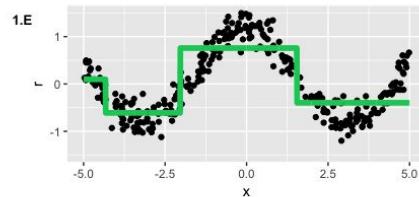
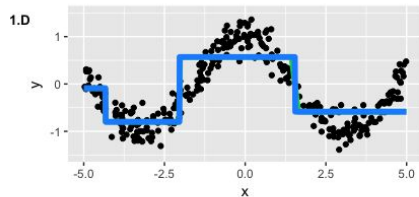
Right: additional tree decisions.

Example by ODS; source:
<https://habr.com/ru/company/ods/blog/327250/>

Gradient boosting: example



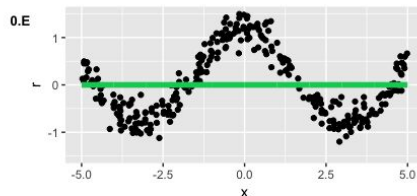
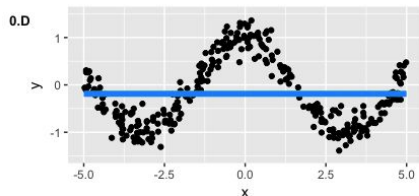
Left: full ensemble on each step.



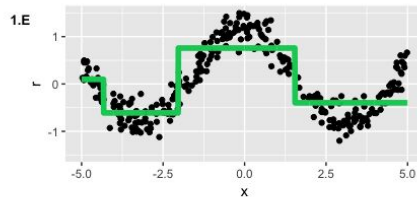
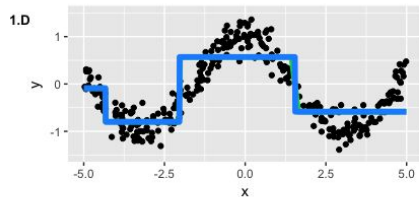
Right: additional tree decisions.

Example by ODS; source:
<https://habr.com/ru/company/ods/blog/327250/>

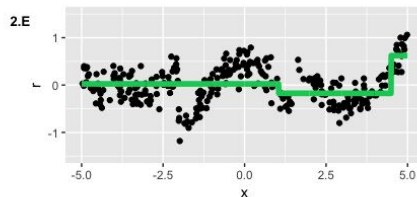
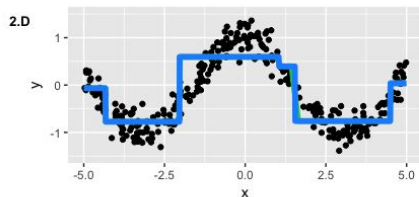
Gradient boosting: example



Left: full ensemble on each step.

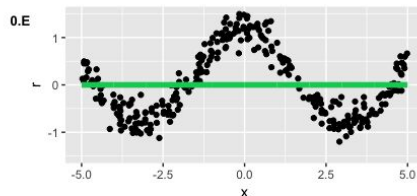
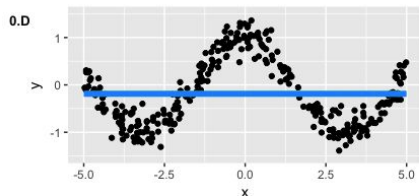


Right: additional tree decisions.

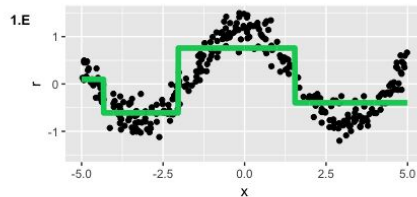
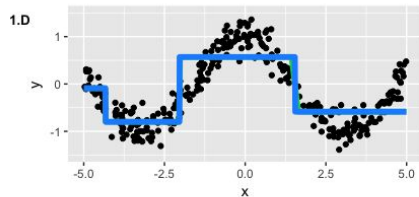


Example by ODS; source:
<https://habr.com/ru/company/ods/blog/327250/>

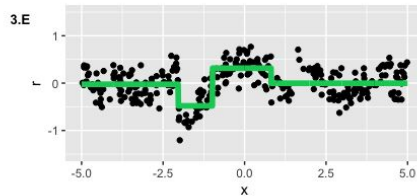
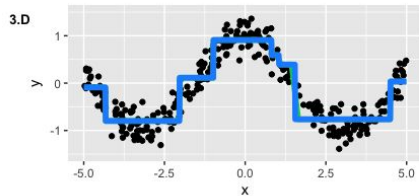
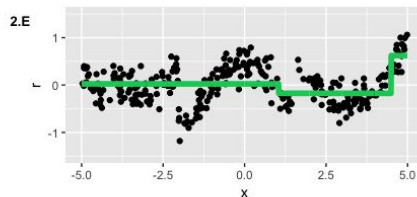
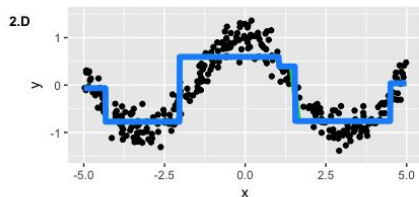
Gradient boosting: example



Left: full ensemble on each step.

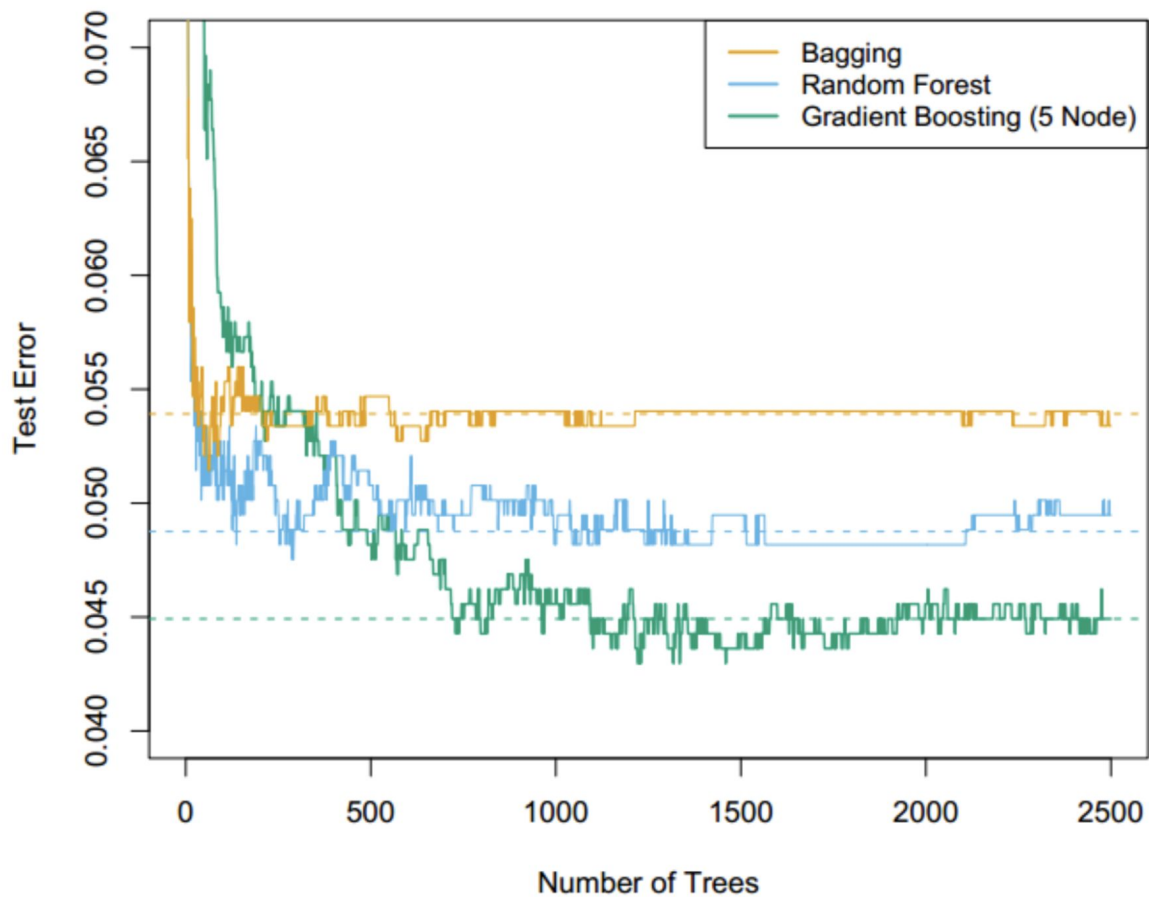


Right: additional tree decisions.

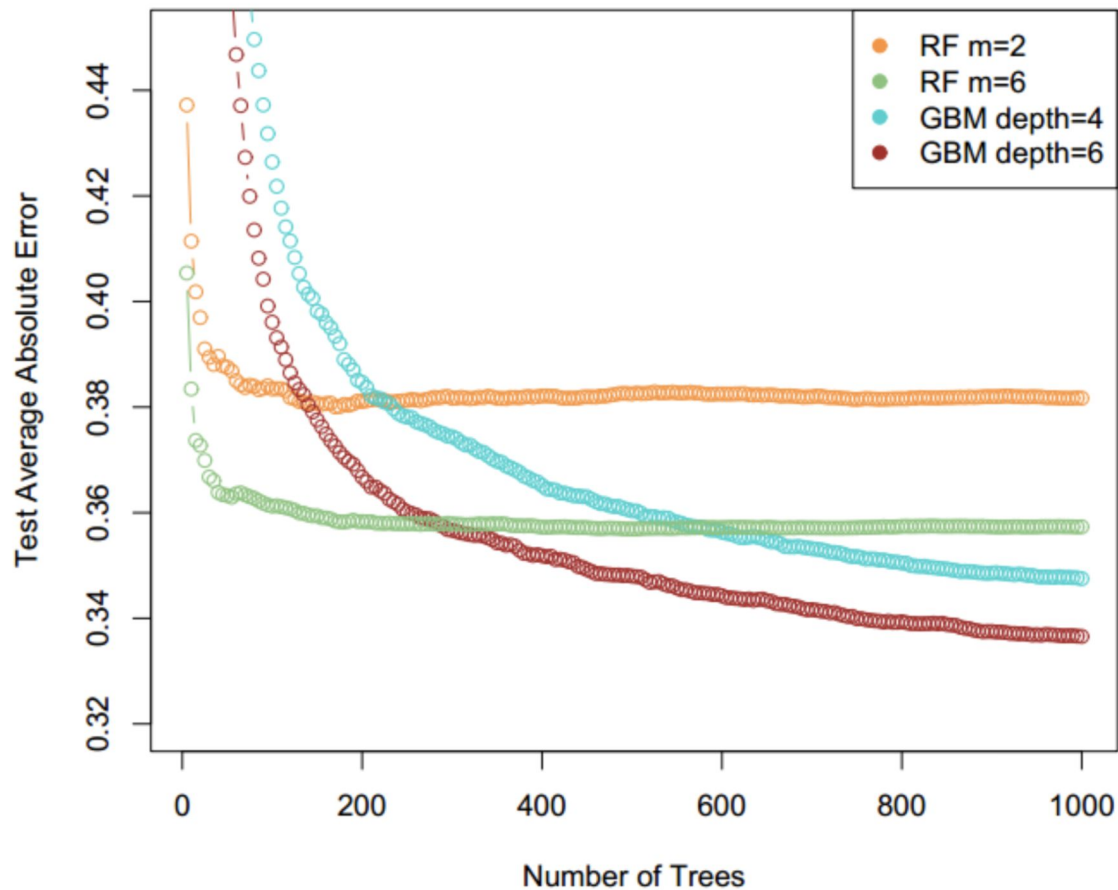


Example by ODS; source:
<https://habr.com/ru/company/ods/blog/327250/>

Spam Data



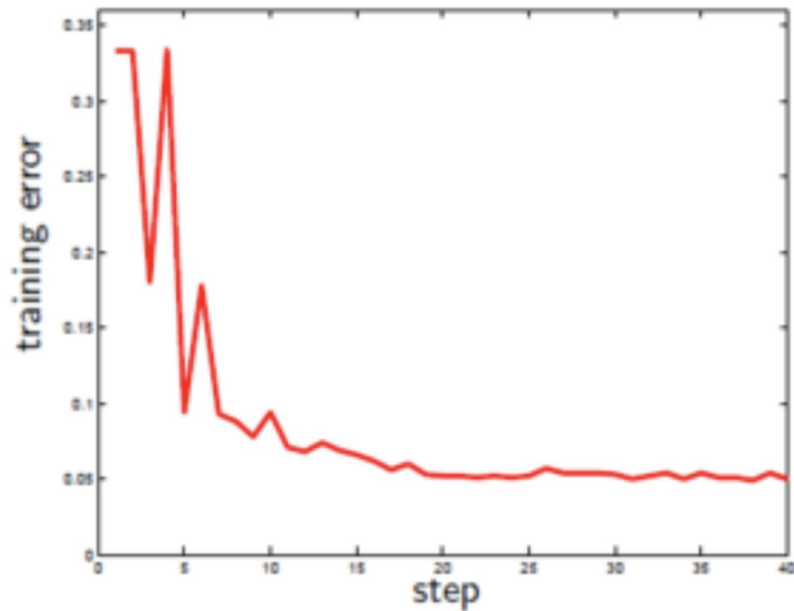
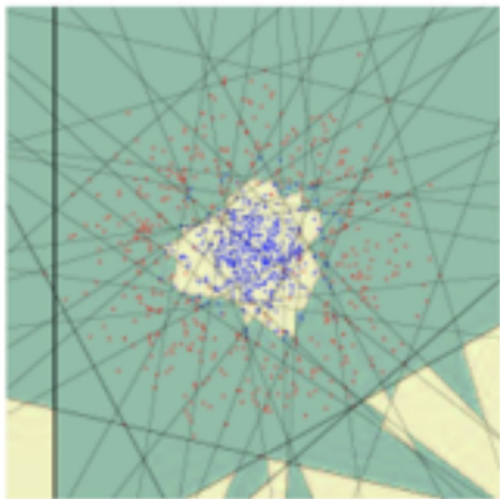
California Housing Data



Boosting with linear classification methods



$t = 40$



Libraries for Gradient Boosting

girafe
ai

04



XGboost by community

- The first leader to arrive!
- Started being developed in 2014
- Best documentation compared to others
- Solves all main tasks: regression, classification, ranking
- Quality is similar to all others
- Nowadays works 20-30% slower than others
- Supports GPU train and inference

<https://xgboost.readthedocs.io/en/stable/>

LightGBM



- Also used to be internal instrument at Microsoft
- Supports

<https://lightgbm.readthedocs.io/en/latest/index.html>

Catboost

Made by Yandex

<https://catboost.ai/>



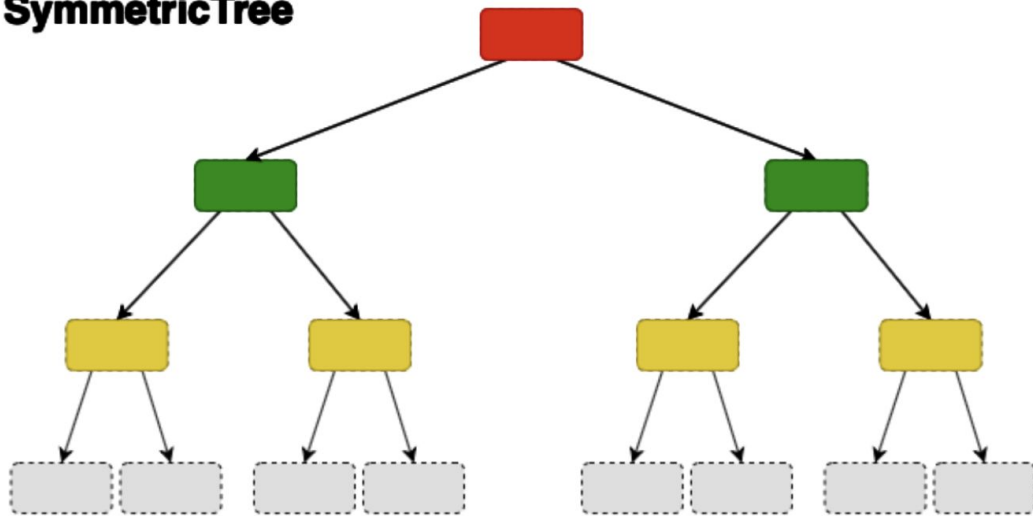
- Used to be internal instrument “Matrixnet”
- Solves all main tasks: regression, classification, ranking
- Tailored to process categorical features out of the box
- Supports GPU train and inference
- Inference bindings for most web languages

Dive to Catboost



- [Explained by core developer for girafe-ai, slides](#)
- <https://apxml.com/courses/mastering-gradient-boosting-algorithms/chapter-6-catboost-gradient-boosting/catboost-oblivious-trees>

SymmetricTree



Main contemporary instruments



1. Catboost by Yandex
 - a. Great default
2. LightGBM by Microsoft
 - a. another good choice
3. XGboost by community
 - a. Great documentation, poor speed

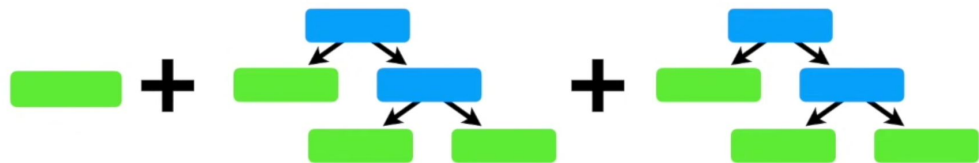
Definitely not sklearn!

Boosting explained in verse!



1. [Boosting explained](#)
2. [XGBoost explained](#)

Gradient Boost Part 1...



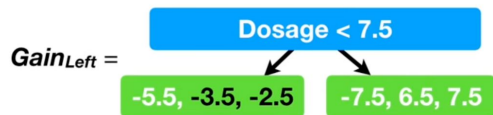
...Regression Main Ideas!!!



Predicted Drug
Effectiveness
0.5

Dosage	Drug Effectiveness	Residuals
5	-5	-5.5
10	-7	-7.5
21	7	6.5
25	8	7.5

Dosage	Drug Effectiveness	Residuals
???	-3	-3.5
???	-2	-2.5



The first **Gain** value, which we will call **Gain_{Left}**, is calculated by putting all of the **Residuals** with missing **Dosage** values into the leaf on the left.

Technical side: training in parallel



Which of the ensembling methods could be parallelized?

- Random Forest: parallel on the forest level (all trees are independent)
- Gradient boosting: parallel on one tree level

Stacking and blending

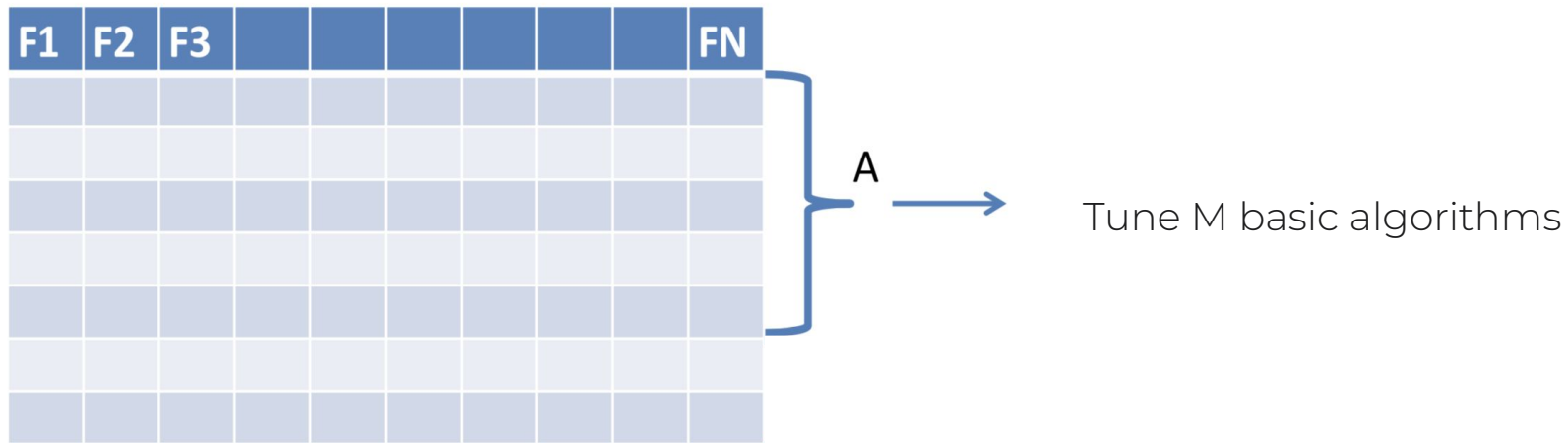
girafe
ai

05



Blending

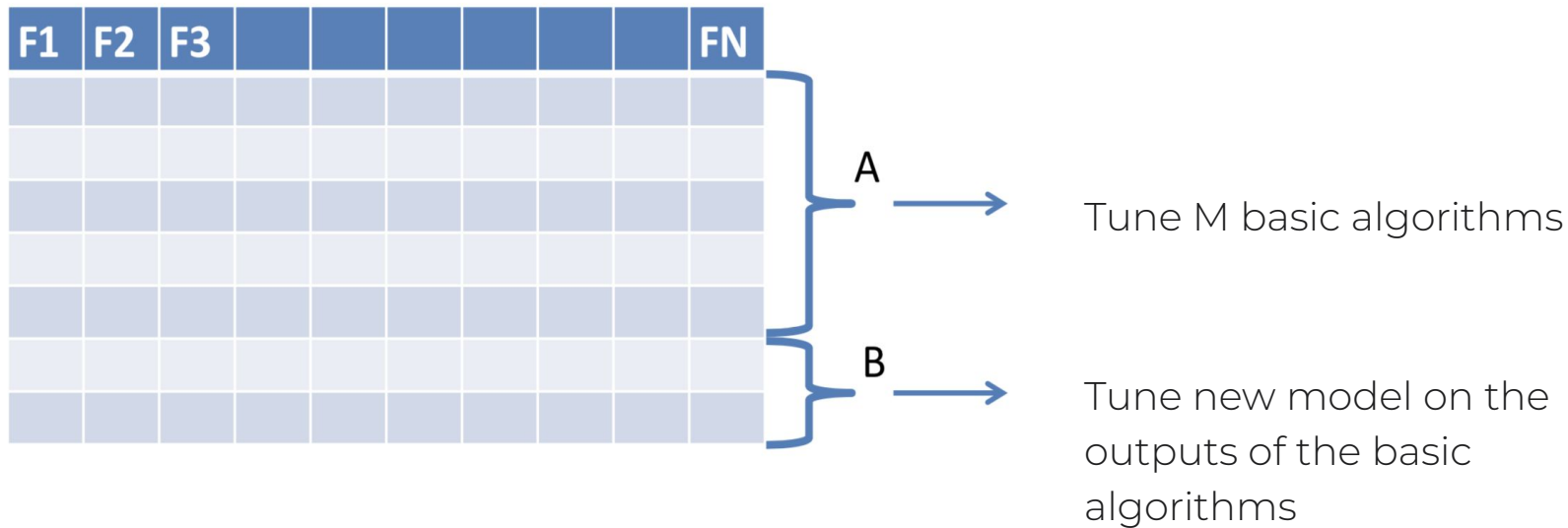
How to build an ensemble from different models?





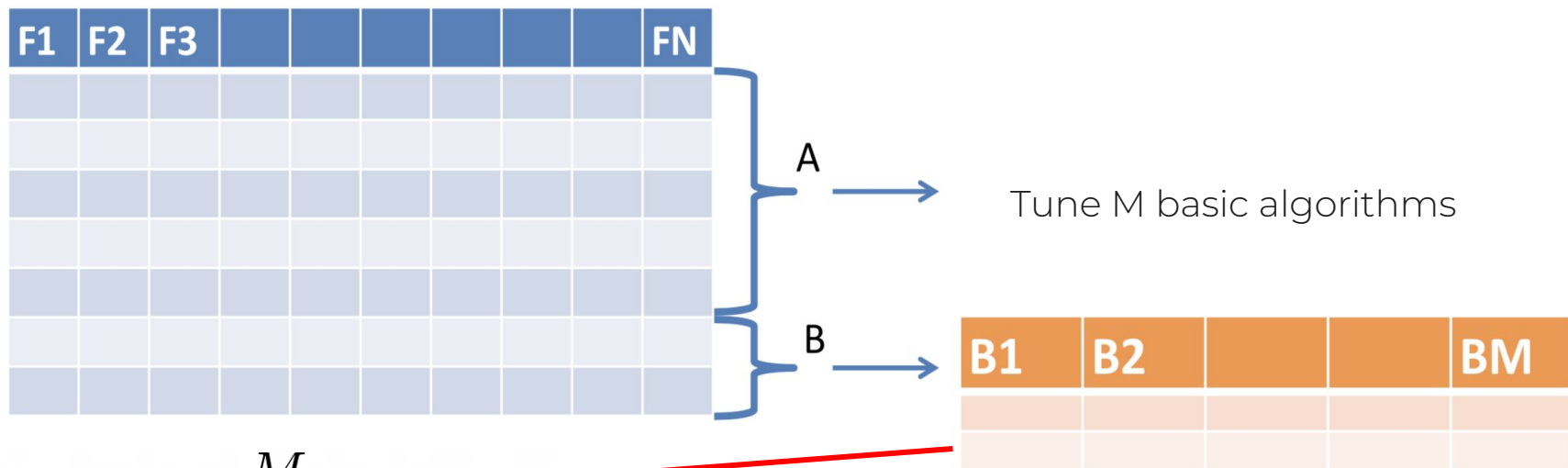
Blending

How to build an ensemble from different models?



Blending

How to build an ensemble from different models?



$$\hat{f}(x) = \sum_{i=1}^M \rho_i f_i(x)$$

$$\sum_{i=1}^M \rho_i = 1, \quad \rho_i \in [0; 1] \quad \forall i$$



Blending

Just combine several strong/complex models.

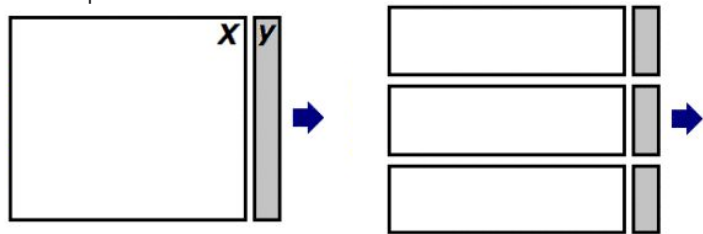
$$\hat{f}(x) = \sum_{i=1}^M \rho_i f_i(x) \quad \sum_{i=1}^M \rho_i = 1, \quad \rho_i \in [0; 1] \quad \forall i$$

- Pros:
 - Simple and intuitive ensembling method.
 - Average several blendings to achieve better results.
- Cons:
 - Linear composition is not always enough.
 - Need to split the data. **How to fix it?**



Stacking

1. Split data into folds



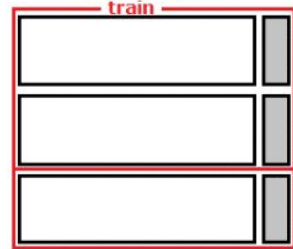
Fit using folds to get meta-features on train



2. Tune models on different groups of folds, predict on left out

3. Tune the new model on the "meta"-features

Fit using all data meta-features on test



Stacking



- Train base algorithm(s) on different groups of folds leaving one fold out.
- Predict the meta-features on the left-out fold and test data.
- Train the meta-algorithm on the meta-features representation of the train data.
- Use it on the meta-features representation of the test data.

Stacking



- Pros:
 - Powerful ensembling method, if you know how to use it
 - Quite popular in ML-competitions
 - One might perform stacking on the meta-features dataset as well
- Cons:
 - Meta-features on each fold are actually predicted by different models
 - However, regularization usually helps
 - Hard to explain your model behaviour

Stacking



Bonus:

Now you know how to stack XGBoost (or CatBoost/LightGBM)



Revise



1. Intuitions
2. Gradient boosting theory
3. Examples
4. Libraries
5. Stacking & Blending

Thanks for attention!

Questions?

